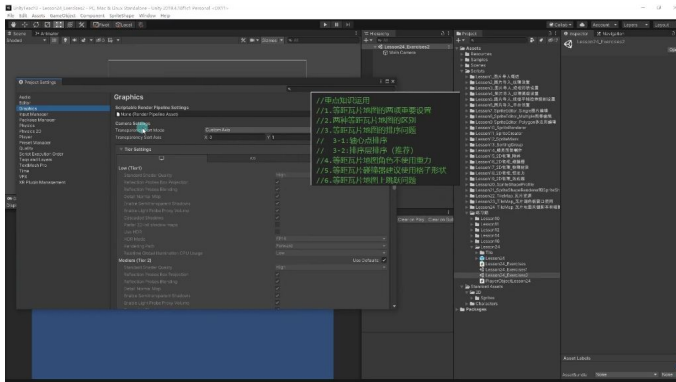


一、练习题二 00:04

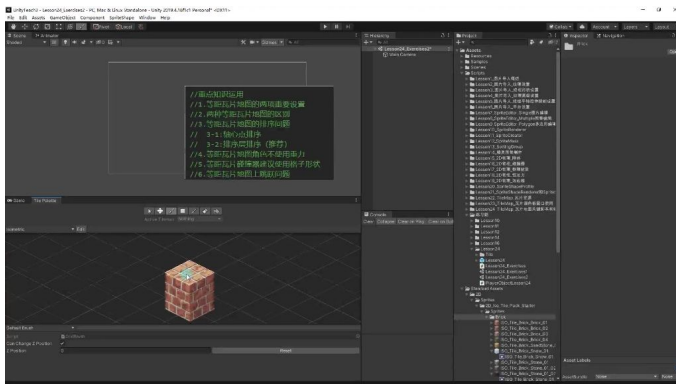
1. 重点知识运用 00:15

1) 等距瓦片地图的两项重要设置 02:06



- 自定义轴设置：
 - 在Project Settings的Graphics选项中，将排序轴从默认轴切换为自定义轴
 - 自定义轴数值设置为：0、1、-0.26（Unity官方规范值）
 - 该设置能解决等距瓦片地图开发时的排序问题
- 渲染模式设置：
 - 将TileMap Renderer上的渲染模式切换为第二种模式
 - 该模式会将格子单独渲染，确保排序正确
 - 适用于所有类型的等距瓦片地图

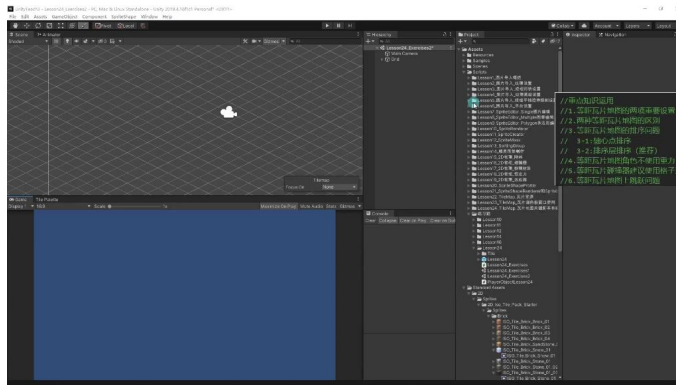
2) 两种等距瓦片地图的区别 03:39



- 普通等距瓦片：
 - 无法通过改变z轴位置调整图片高度
 - 即使开启"Can Change Z Position"功能也无效
 - 排序问题较少，推荐使用
- Z as Y等距瓦片：
 - 可以通过加减号改变z轴位置，实现图片上移/下移效果
 - 允许通过y轴模拟z轴的纵深感
 - 可能出现意外的排序问题
- 使用建议：
 - 普通等距瓦片更稳定，推荐作为主要选择
 - 若使用Z as Y类型，需特别注意排序问题
 - 两种类型在调色板窗口中的创建方式相同

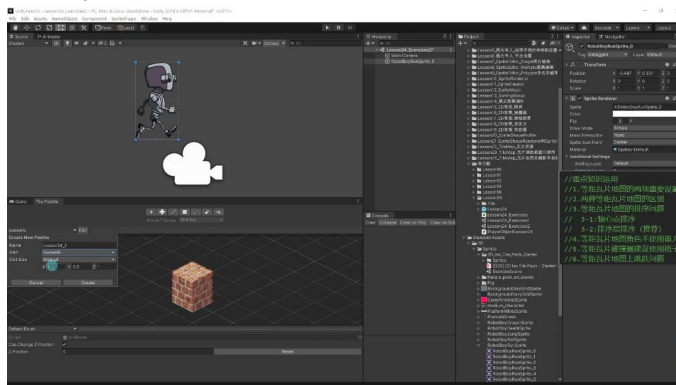
2. 等距瓦片地图制作

1) 基础设置



- 轴心点设置：角色资源需要调整轴心点(Pivot)到脚下位置，通常设置为(0.5,0)，这样在等距地图中才能正确排序
- 资源导入：导入图片资源时要注意纹理类型(Texture Type)设置为Sprite(2D and UI)，并正确设置像素每单位(Pixels Per Unit)

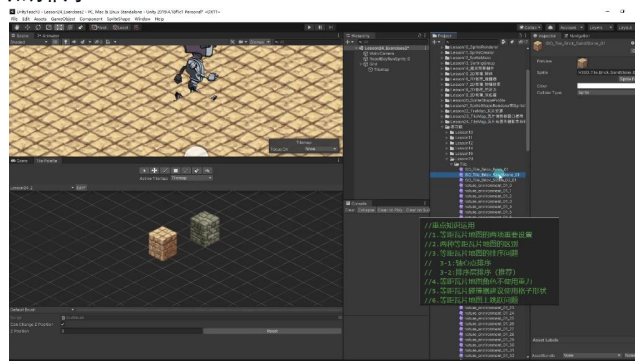
2) 瓦片地图创建



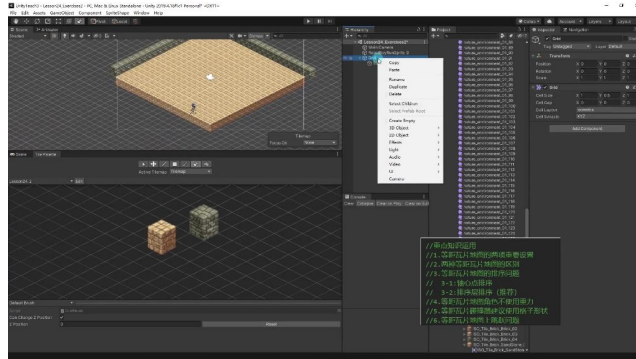
- 创建步骤：
 - 右键创建2D对象 → Tilemap → Isometric
 - 新建调色板(Palette)并保存
 - 选择菱形瓦片资源拖入调色板
- 重要设置：
 - 瓦片大小默认为1×0.5的菱形
 - 渲染模式(Render Mode)需设置为Chunk

3) 排序问题解决方案

- 轴心点排序



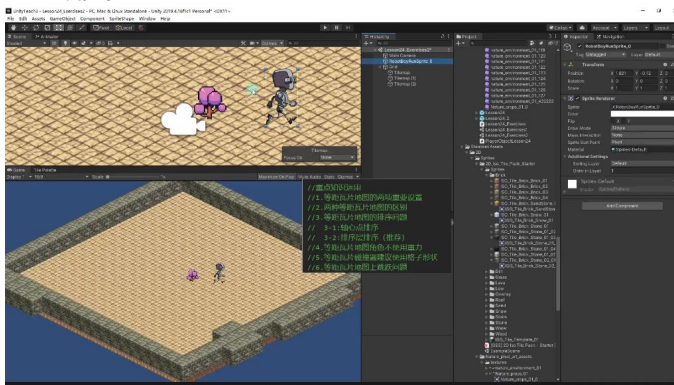
- 原理：通过比较角色和瓦片的轴心点y值决定前后关系
- 问题：修改轴心点会影响瓦片与菱形的匹配关系，导致编辑困难
- 应用场景：适合简单场景，不推荐复杂地图使用
- 排序层排序(推荐)



- 实现方法：
 - 创建多个Tilemap层
 - 设置不同Sorting Layer和Order in Layer
 - 地面层设为0，角色层1，遮挡层设为2
 - 优势：编辑直观，排序稳定
 - 混合使用：对于需要动态遮挡的物体(如树木)，可结合轴心点排序
- 4) 碰撞与物理设置
- 碰撞器选择：建议使用格子形(Grid)碰撞器而非精确碰撞器
 - 重力设置：角色应禁用重力，通过脚本控制移动
 - 跳跃问题：等距地图的跳跃需要特殊处理z轴坐标
- 5) 实用技巧
- 美术资源规范：
 - 地面资源轴心点在底部中心
 - 墙面资源轴心点在顶部中心
 - 可遮挡物(如树木)轴心点在底部
 - 编辑技巧：使用填充工具快速创建大面积地面，再用笔刷细化
- 6) 常见问题解决
- 显示错位：检查瓦片资源的轴心点是否与菱形匹配
 - 排序异常：确认Tilemap的渲染模式为Chunk
 - 遮挡错误：合理使用多层Tilemap和Sorting Layer组合

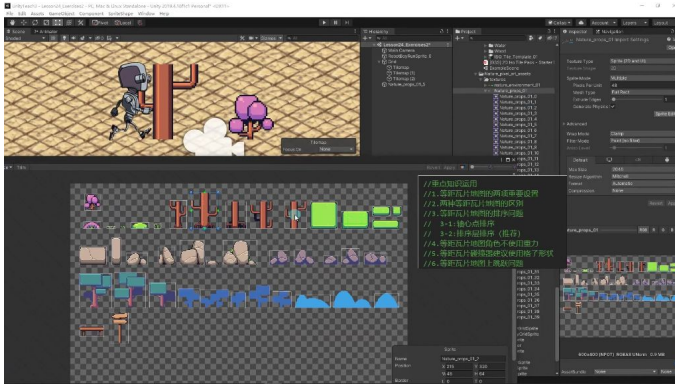
二、场景移动 24:10

1. 轴心点排序 24:30



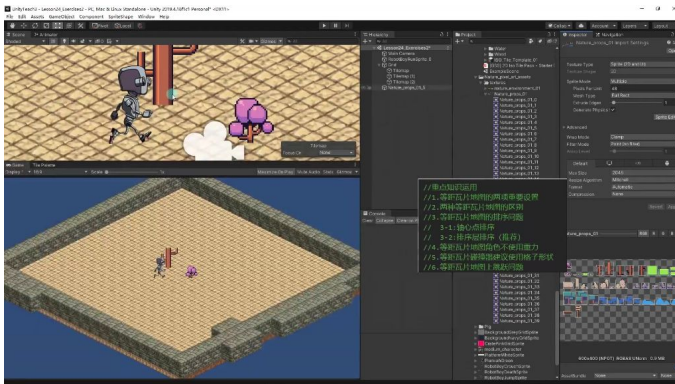
- 同层排序原理：当对象轴心点在下方时显示在前，在上方时会被遮挡
- 操作演示：移动玩家对象时，通过调整轴心点位置可控制显示优先级
- 适用场景：适用于需要同层对象间产生前后关系的排序需求

2. 2D对象排序 24:51



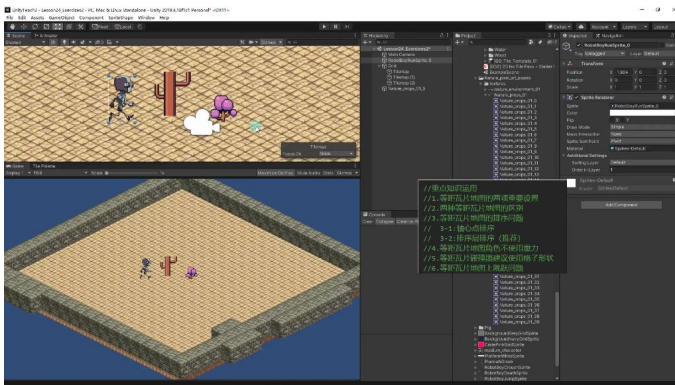
-
- 添加方法：直接拖拽2D对象到场景中，与玩家对象进行交互排序
- 层级设置：需将对象层级改为与玩家相同的层级（如第一层）
- 初始问题：新添加的2D对象可能出现排序层显示异常

3. 轴心点设置的重要性 26:34



-
- 关键步骤：必须将排序点设置为轴心点而非中心点
- 编辑方法：
 - 选择需要排序的2D对象
 - 进入编辑模式调整轴心点位置（推荐底部中心点）
 - 应用修改使轴心点生效
- 效果验证：修改后对象位置会发生变化，移动时能正确产生前后关系

4. 排序方法的选择 26:57

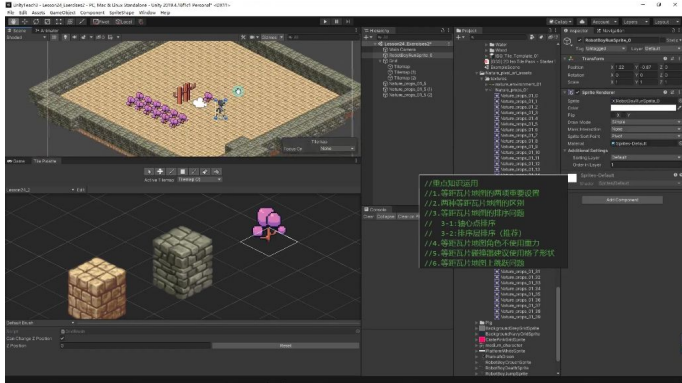


-
- 瓦片地图方式：
 - 优点：编辑快捷，可批量添加（如一次拖拽多棵树）
 - 缺点：每个菱形格子只能放置一个对象
- 精灵对象方式：
 - 优点：不受格子限制，可密集排列（如一个格子放多棵树）
 - 缺点：需要逐个设置轴心点

5. 精灵对象排序的优缺点 27:38

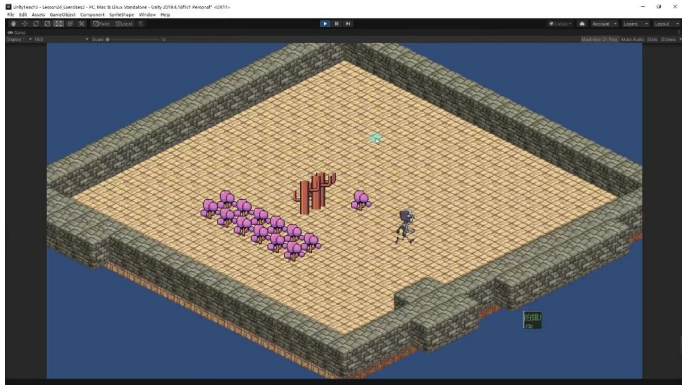
- **密集排列演示：**使用Ctrl+D复制多个对象，可实现在小范围内密集分布
- **视觉效果：**能营造出树林密布的环境感，玩家移动时前后关系变化自然
- **选择建议：**根据实际场景需求决定使用瓦片地图方式还是精灵对象方式

三、等距瓦片地图设置与角色控制



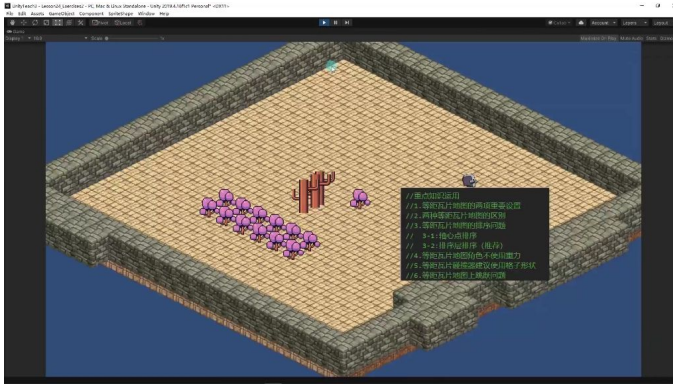
-
- 1. 等距瓦片地图的重要设置
 - **两项关键设置：**
 - 轴心点排序（Pivot Point Sorting）
 - 排序层设置（Sorting Layer）
 - **两种地图区别：**
 - 传统等距地图：使用固定视角
 - 伪Z轴地图：允许纵深移动，需要特殊碰撞处理

2. 角色物理组件设置



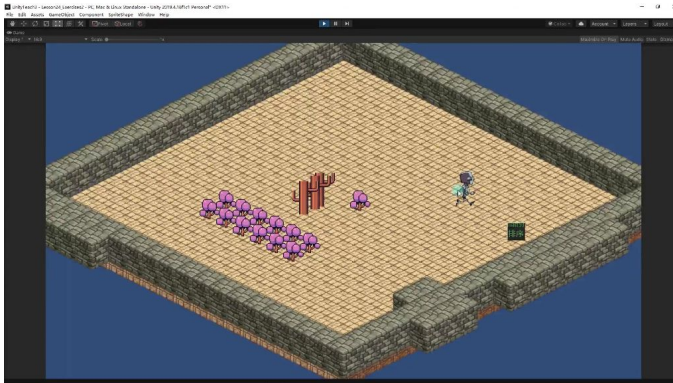
-
- **碰撞器选择：**
 - 推荐使用胶囊碰撞器2D（Capsule Collider 2D）
 - 需调整碰撞器大小匹配角色尺寸
- **刚体特性：**
 - 必须添加Rigidbody 2D组件
 - **重力设置：**必须设为0（禁用重力）
 - **角度约束：**需冻结Z轴旋转防止角色倾斜

3. 碰撞系统实现



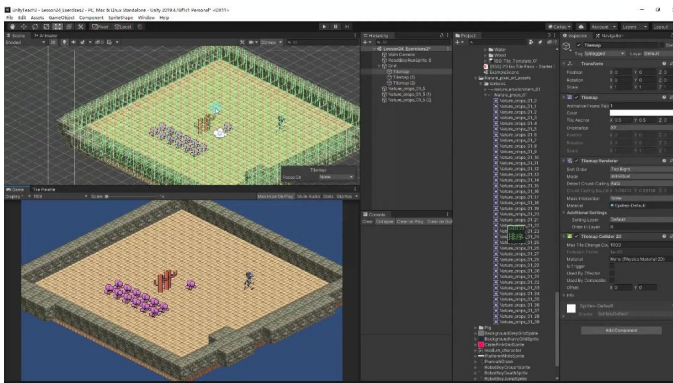
- 地形碰撞规则：
 - 仅边缘区域需要碰撞体
 - 可通行区域必须保持无碰撞
 - 碰撞形状建议：使用格子形状（Grid-based）碰撞器
- 角色移动控制：
 - 通过Transform直接控制位移（非物理移动）
 - 跳跃功能需特殊处理（不依赖物理系统）

4. 瓦片碰撞生成技巧



- 资源设置：
 - 需关闭瓦片资源的默认碰撞生成
 - 通过Tile Collider 2D手动添加必要碰撞
- 常见问题：
 - 角色卡顿：因碰撞体形状与视觉不匹配导致
 - 解决方案：使用简化碰撞形状，避免复杂轮廓

5. 实现要点总结

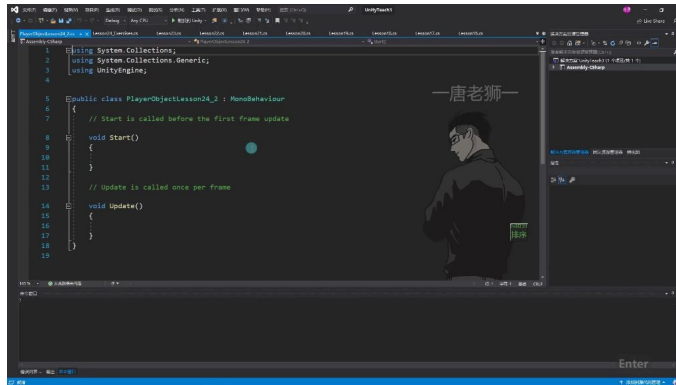


- 核心原则：
 - 伪3D效果通过2D系统模拟实现
 - Z轴深度通过Y坐标偏移表现

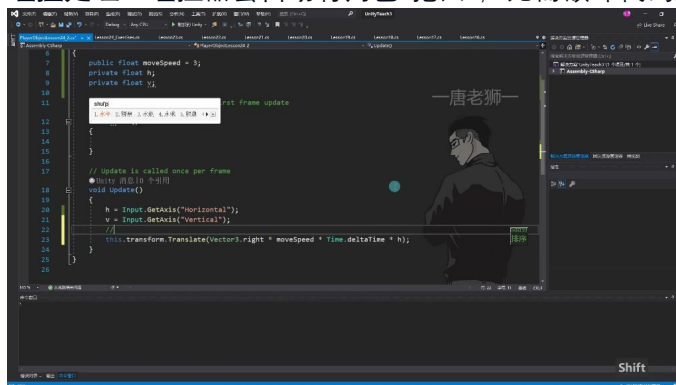
- 优化建议：
 - 使用Sorting Layer控制渲染顺序
 - 碰撞体应尽量简化以提高性能
 - 角色控制器需完全通过脚本实现物理模拟

四、瓦片地图练习 31:38

1. 编写脚本控制玩家移动 33:10

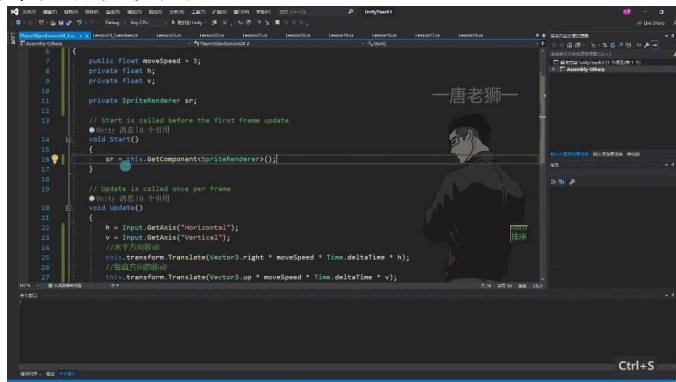


- 移动控制原理：通过Input.GetAxis获取水平和垂直输入值，使用transform.Translate实现移动
- 速度设置：public float moveSpeed = 3 控制移动速度
- 移动计算：
 - 水平移动：Vector3.right * moveSpeed * Time.deltaTime * h
 - 垂直移动：Vector3.up * moveSpeed * Time.deltaTime * v
- 碰撞处理：碰撞器会自动将角色"拖回"，无需额外代码处理

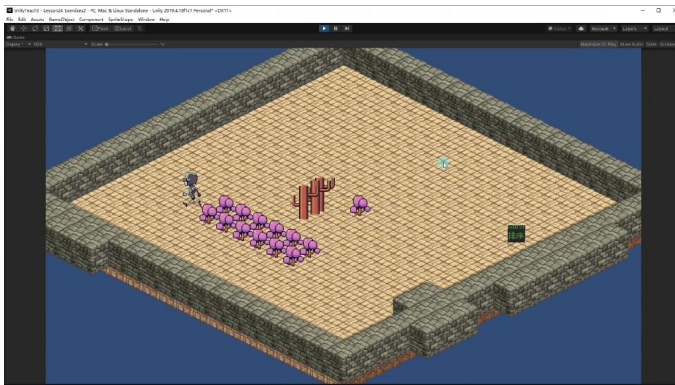


- 输入监听：使用h和v两个私有变量存储输入值
- 帧更新机制：在Update()中持续检测输入并更新位置
- 物理特性：2D游戏中不使用重力，仅通过碰撞器限制移动范围

2. 脚本控制玩家转向 36:32



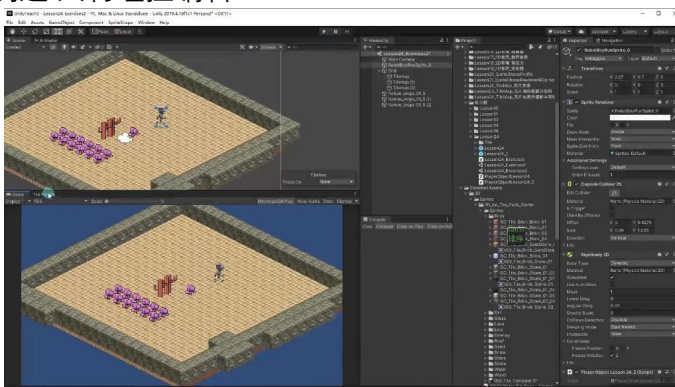
- 组件获取：通过GetComponent<SpriteRenderer>()获取精灵渲染器
- 翻转逻辑：
 - h<0时设置sr.flipX = true（向左转）
 - h>0时设置sr.flipX = false（向右转）
- 状态保持：h=0时不改变当前翻转状态



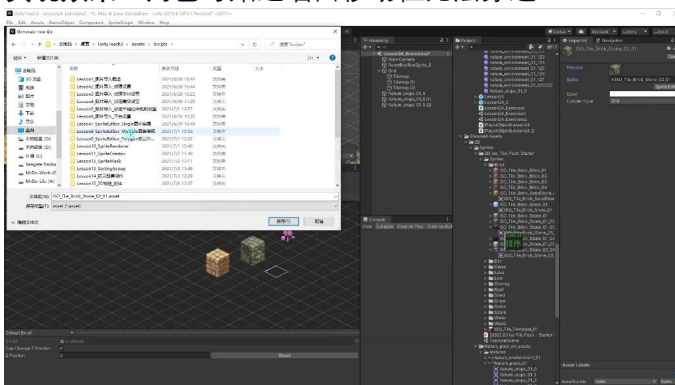
- 视觉表现：翻转后配合动画可实现奔跑效果
- 无重力影响：纯2D移动不涉及物理重力系统
- 碰撞效果：与瓦片地图碰撞器自然交互

3. 应用案例 37:57

1) 例题:人物碰撞编辑

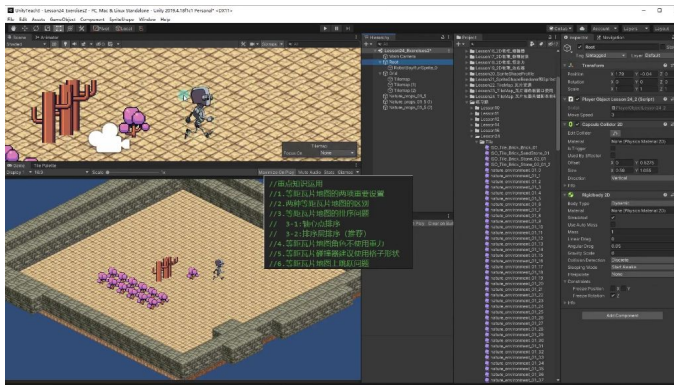


- 碰撞形状选择：
 - 地面不需要碰撞器（Collider Type设为None）
 - 围墙使用Grid格子形状而非Sprite形状
- 碰撞区域优化：
 - 内部瓦片使用无碰撞版本
 - 外围延伸1-2层格子作为碰撞区域
- 实现效果：角色可贴近墙面移动但无法穿过

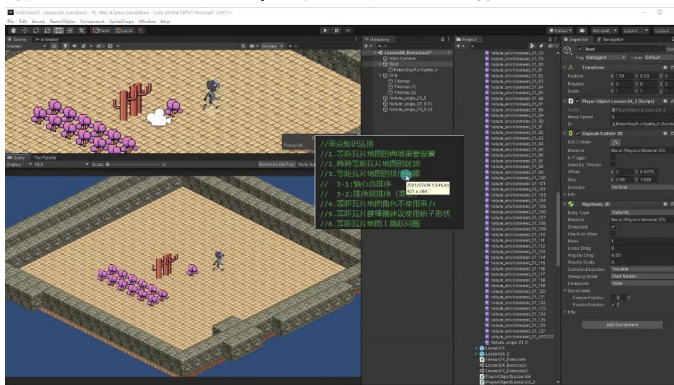


- 资源处理：同一瓦片资源创建两个版本（带/不带碰撞）

- 层级管理：确保选择正确的Tilemap层级进行编辑
 - 批量填充：使用填充工具快速设置外围碰撞区域
- 2) 例题:等距瓦片地图跳跃问题 41:35

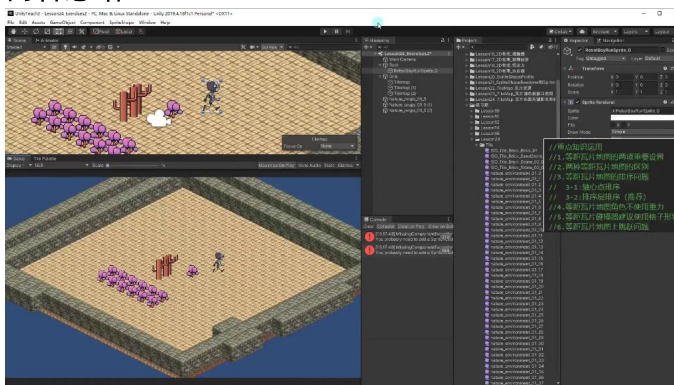


- 层级结构：
 - 根物体 (Root) 包含刚体和碰撞器
 - 子物体仅包含视觉表现 (Sprite)
- 跳跃原理：
 - 控制子物体本地坐标y轴移动
 - 根物体保持不动，仅处理x轴移动
- 落地判断：当子物体y坐标归零时视为落地



- 脚本调整：将控制脚本移至根物体
- 组件分配：
 - 刚体和碰撞器保留在根物体
 - 子物体仅保留SpriteRenderer
- 输入监听：通过空格键触发跳跃动作

五、内容总结 44:40



- 等距瓦片设置：
 - 两项重要设置：轴心点和排序方式

- 两种地图类型区别：规则/不规则等距
- **排序方案：**
 - 轴心点排序（简单但不精确）
 - 排序层排序（推荐方案）
- **物理特性：**
 - 角色不使用重力
 - 碰撞器建议使用Grid格子形状
- **特殊处理：**
 - 跳跃通过控制子物体y轴实现
 - 碰撞区域需要特殊设计

六、知识小结

知识点	核心内容	易混淆点/注意事项	应用场景
尾Z轴概念	通过y轴模拟z轴纵深效果，实现2D游戏的伪3D视觉	与真实3D空间z轴的区别	等距视角2D游戏开发
等距瓦片设置	1. Project Settings中设置自定义排序轴(0,1,-0.26) 2. Tilemap Renderer切换为Individual模式	默认轴与自定义轴的区别	解决等距地图渲染排序问题
瓦片类型对比	普通等距瓦片 vs Z as Y瓦片（支持z轴位移调整）	Z as Y瓦片可能导致排序异常	常规开发推荐使用普通等距瓦片
轴心点设置	1. 角色轴心点需设置在脚底 2. 地形瓦片轴心点需匹配菱形单元格	美术资源轴心点与单元格匹配关系	实现正确的深度排序
排序系统	1. 通过Sorting Layer分层 2. 同层对象使用Sprite Sort Point(轴心点)排序	中心点排序与轴心点排序的区别	处理复杂场景遮挡关系
碰撞生成	1. 地面不生成碰撞	Sprite形状碰撞与Grid碰撞的效果差异	实现符合视觉的物理阻挡

	2. 墙体使用Grid模式生成碰撞		
角色移动	Transform平移控制（禁用重力）	与刚体物理移动的区别	尾Z轴地图的特殊移动需求
跳跃实现	通过子物体y轴位移模拟跳跃效果	根物体保持平面移动逻辑	保持2D移动同时实现3D跳跃视觉效果
多层地图	1. 背景层 (Sorting Order=0) 2. 角色层(1) 3. 前景遮挡层 (≥ 2)	不同层级间的渲染优先级	构建复杂场景层次关系